Probabilistic Regression Identifiability — Research Notes

ClassReg vs ProbReg, the head-index swap degeneracy, and three candidate fixes

Apr 2026 session

2026-04-21

Contents

1. Executive summary **2**

2. Background: architecture and the identifiability problem **2**
2.1 ProbReg SEP_HEADS — what the model looks like . . . 2
2.2 Why this is degenerate: head-index swap . . . 3
2.3 Why ClassReg doesn't have this . . . 3

3. Three orthogonal fixes (hypotheses) **3**
3.1 MDN NLL . . . 3
3.2 CE_STOP_GRAD . . . 4
3.3 Anchored heads . . . 4
3.4 How the three are orthogonal . . . 5

4. Experimental design **5**
4.1 The 8-cell matrix . . . 5
4.2 Datasets (all $n = 600$–800, $d = 1$) . . . 6
4.3 Grid . . . 6
4.4 Metrics (measured on test split) . . . 6

5. Raw results — full 8-cell × 4-dataset × 2-k table **6**

6. Visual interpretation — what the probability-curve pages show **8**
6.1 `h_i(p_i)` plots — head-index anchoring . . . 8
6.2 `p_i(x)` plots — classifier signature . . . 8
6.3 What `max_p_mid` tells us at $k = 5$ . . . 9

7. Analysis and conclusions **9**
7.1 Anchored heads — recommend ON by default . . . 9
7.2 ClassReg vs ProbReg . . . 9
7.3 CE_STOP_GRAD — situational dial, not default . . . 10
7.4 MDN NLL — situational, not uniform . . . 10
7.5 Recommendation matrix . . . 10
7.6 The best "safe default" overall . . . 10

8. The symlog hypothesis — a negative result **11**

8.1 The diagnosis we proposed . . . 11
8.2 The experiment . . . 11
8.3 Results . . . 11
8.4 What it rules out and what it leaves . . . 12
8.5 Remaining suspects for the C/D failure . . . 12

9. Open questions and next experiments **12**
9.1 Hybrid opt-strategy (most promising) . . . 12
9.2 Adaptive bin boundaries on heavy-tail y . . . 13
9.3 Per-head log-variance warm start . . . 13
9.4 Anchored + MDN + symlog on real domains . . . 13
9.5 Re-test Cell D with classifier warm-start . . . 13
9.6 Larger k sweep on exponential . . . 13

10. Bug log (during this investigation) **13**
10.1 SINGLE_HEAD_FINAL_OUTPUT + dynamic-k shape mismatch (`280ad1f`) . . . 13
10.2 `calculate_class_value_ranges` numpy.float32 rejection (`19bdacc`) . . . 13
10.3 BatchNorm with batch size 1 (`fe63c90`) . . . 14
10.4 `final_results_report.py` column name (`fe63c90`) . . . 14
10.5 B1 — monotonic head init with `mean=-3.0` (**pre-existing, not fixed**) . . . 14

11. File, artefact, and commit pointers **14**

12. What we'd pick up first in a future session **15**

1. Executive summary

This document is the durable record of our investigation into the **head-index swap degeneracy** in ProbReg's `SEPARATE_HEADS` parametrization and the three orthogonal fixes we tested: an MDN-style likelihood, classifier–head gradient detachment (`CE_STOP_GRAD`), and anchored heads. It covers the theoretical motivation, the full 8-cell experimental matrix, the raw results across four toy datasets, visual interpretations of the probability curves, the recommendation we would take forward, a negative-result side experiment (the symlog hypothesis for heavy-tail targets), open questions, and a log of bugs found during this work. The goal is that a future session can resume without re-deriving the context.

Headline finding. Anchored heads solve the head-swap degeneracy cleanly at essentially no MSE cost and should become the default. CE_STOP_GRAD and MDN are situationally useful dials, not uniform improvements. ClassReg is consistently beaten by ProbReg except on narrow-range symmetric targets. A single negative result: on heavy-tail targets (exponential), CE_STOP_GRAD cells fail by an order of magnitude and **target-range compression via `symlog` does not rescue them** — the root cause is something other than target scale.

2. Background: architecture and the identifiability problem

2.1 ProbReg SEP_HEADS — what the model looks like

For a regression target $y \in \mathbb{R}$ and input $x \in \mathbb{R}^d$, ProbReg in the `SEPARATE_HEADS` configuration binarises y into k percentile-based bins and learns:

- A classifier $q_\phi(x) = \text{softmax}(\text{NN}_\phi(x)) \in \Delta^{k-1}$ producing class probabilities $p_i(x)$ for $i = 1, \ldots, k$.

- k regression heads $h_i(p_i; \psi_i) \in \mathbb{R}^2$, each returning a per-class mean μ_i and log-variance $\log \sigma_i^2$.

The model's predictive mean and variance are combined via the **law of total variance** (LTV):

$$\mu_{\text{total}}(x) \;=\; \sum_{i=1}^{k} p_i(x)\, \mu_i(p_i(x)),$$

$$\sigma^2_{\text{total}}(x) \;=\; \underbrace{\sum_{i=1}^{k} p_i(x)\, \sigma_i^2(p_i(x))}_{\text{within-class}} \;+\; \underbrace{\sum_{i=1}^{k} p_i(x)\, \mu_i^2(p_i(x)) - \mu^2_{\text{total}}(x)}_{\text{between-class}}.$$

The training loss is the Gaussian NLL with these two moments:

$$\mathcal{L}_{\text{LTV}}(y, x) \;=\; \tfrac{1}{2}\left(\log(2\pi\, \sigma^2_{\text{total}}) + \frac{(y - \mu_{\text{total}})^2}{\sigma^2_{\text{total}}}\right).$$

2.2 Why this is degenerate: head-index swap

The loss $\mathcal{L}_{\text{LTV}}$ depends only on $(\mu_{\text{total}}, \sigma^2_{\text{total}})$. For a fixed input x with $p_i(x) \to 1$, the total mean collapses to a single head output:

$$\lim_{p_i \to 1} \mu_{\text{total}}(x) \;=\; \mu_i(p_i = 1).$$

There is **no pressure in the loss** to make μ_i equal the centroid c_i of bin i (where $c_i = \mathbb{E}[y \mid y \in \text{bin}_i]$ from training data). Any consistent relabeling $\sigma \in S_k$ with $p'_i = p_{\sigma(i)}$, $\mu'_i = \mu_{\sigma(i)}$ yields the same loss. Classifier "probs" therefore do not have to match any specific bin — the mapping between head index and y-value is free.

2.3 Why ClassReg doesn't have this

In `ClassifierRegressionModel`, the classifier is supervised by cross-entropy against **hard bin labels** derived from y. The mapper $m_i \mapsto \hat{y}$ (e.g. `LOOKUP_MEDIAN`) is data-driven, not learned via the regression loss. The hard labels pin head-index $\leftrightarrow$ bin-of-y exactly; there is no swap freedom. The cost is that the classifier must commit to a hard class structure, which hurts smooth regression.

3. Three orthogonal fixes (hypotheses)

3.1 MDN NLL

Replace the LTV Gaussian NLL with a mixture-density-network likelihood where probabilities enter the likelihood structurally:

$$\mathcal{L}_{\text{MDN}}(y, x) \;=\; -\log \sum_{i=1}^{k} p_i(x)\, \mathcal{N}(y; \mu_i, \sigma_i^2).$$

Because each component's likelihood scales with its own p_i, a relabeling changes the per-sample log-likelihood — the loss is **not** invariant under head-index swaps. Identifiability comes from the likelihood structure itself.

Implementation note (`automl_package/utils/losses.py:mdn_nll`): the naive evaluation of $\log \sum_j p_j\, \mathcal{N}_j$ is numerically unstable (both factors can underflow). The code computes $\log p_j$ and $\log \mathcal{N}_j$ separately and combines them via `torch.logsumexp` with an ϵ-floor on p_j, which is the standard Bishop 1994 form.

3.2 CE_STOP_GRAD

Supervise the classifier directly by cross-entropy against bin labels, and `detach()` the classifier's probabilities before they flow into the regression heads:

$$\mathcal{L}_{\mathrm{CE}}(x) \;=\; -\sum_{i=1}^{k} \mathbb{1}[y \in \mathrm{bin}_i]\, \log p_i(x), \qquad \tilde{p}_i = p_i.\mathrm{detach}().$$

Total loss $\mathcal{L}_{\mathrm{CE}} + \mathcal{L}_{\mathrm{LTV}}(\tilde{p}, \mu, \sigma)$.

Although the total loss is arithmetically a sum, the stop-gradient severs the gradient path: the classifier receives gradient only from $\mathcal{L}_{\mathrm{CE}}$ (because $\tilde{p}$ is detached before the regression module), and the heads receive gradient only from $\mathcal{L}_{\mathrm{LTV}}$ (because they do not appear in $\mathcal{L}_{\mathrm{CE}}$). In `SEPARATE_HEADS` the classifier and heads share no parameters, so the two losses optimize disjoint parameter sets: the training dynamic is effectively two uncoupled optimizations sharing a forward pass. The classifier is pinned by hard labels (like ClassReg), and the heads fit the regression loss treating $\tilde{p}$ as a frozen input signal. Identifiability comes from hard bin supervision. This lack of co-adaptation is also the leading suspect for the exponential failure (§8.5).

3.3 Anchored heads

Reparametrise each head so its output **at $p_i = 1$ is structurally the bin centroid**:

$$h_i(p_i) \;=\; c_i \;+\; (1 - p_i)\, f_i(p_i),$$

where c_i is the precomputed mean of y in bin i and f_i is a small MLP. At $p_i = 1$, $h_i = c_i$ exactly, with no gradient path for deviation. For $p_i < 1$, the residual f_i retains full expressivity. Identifiability comes from a hard structural prior rather than loss structure.

The centroid target is wrong for edge bins — a real limitation. The head returns the per-class mean μ_i and the model combines them by LTV: $\mu_{\mathrm{total}}(x) = \sum_j p_j(x)\, \mu_j(p_j(x))$. When $p_i(x) \to 1$, $\mu_{\mathrm{total}}(x) \to h_i(1)$. Under MSE loss the optimal target for $h_i(1)$ is

$$h_i^\star(1) \;=\; \mathbb{E}\big[y \,\big|\, p_i(x) = 1\big].$$

This is conditioned on the **classifier's confidence**, not on bin membership. It is *not* equal to $c_i = \mathbb{E}[y \mid y \in \mathrm{bin}_i]$ in general — $\{x : p_i(x) = 1\}$ is typically a strict subset of $\{x : y \in \mathrm{bin}_i\}$, specifically the "deep interior" away from bin boundaries.

For edge bins the two expectations differ substantially:

- **Upper bin** ($i = k-1$): samples with $p_i(x) = 1$ live in the deep upper interior close to $y_{\max}$. So $\mathbb{E}[y \mid p_{k-1} = 1] > c_{k-1}$. The correct anchor is higher than the centroid.
- **Lower bin** ($i = 0$): symmetric. $\mathbb{E}[y \mid p_0 = 1] < c_0$.
- **Inner bins**: bin distribution is (approximately) symmetric about c_i so $\mathbb{E}[y \mid p_i = 1] \approx c_i$ and the centroid anchor is fine.

Anchoring $h_i(1) = c_i$ therefore **systematically pulls edge-bin predictions toward the interior** — downward on the upper tail, upward on the lower tail. On heavy-tailed targets this is harmful; the data confirms it (Table in §5, exponential $k = 5$: anchored Cell B MSE = 1.84 vs unanchored Cell A MSE = 0.50).

For contrast, ClassReg with `LOOKUP_MEDIAN` does not have this problem: its mapper is built empirically from observed (p, y) pairs, effectively estimating $\mathbb{E}[y \mid p_i(x)]$ (or the median thereof) directly from training data rather than from the centroid.

Interaction with monotonic constraints — worse still. The `use_monotonic_constraints=True` option replaces the head's Linear layer with `monotonic_linear`, which uses `softplus` weights so that h_i is monotone non-decreasing (negated for `Monotonicity.NEGATIVE`). With anchor active:

- Upper class (monotone positive): maximum at $p_i = 1$, pinned to c_i, so $h_i(p_i) \le c_i$ for all p_i. The model cannot predict above c_{k-1} anywhere.
- Lower class (monotone negative): $h_0(p_0) \ge c_0$ always.

The model's output range is bounded inside $[c_0, c_{k-1}]$ — the tails of y outside this interval are structurally unreachable. This is a hard architectural limit for unbounded/heavy-tail targets. The identifiability sweep disabled `use_monotonic_constraints` so this combination was not stressed, but it is an important caveat for any downstream use.

Refinements to the anchor target. The centroid is the wrong target for edge bins. Options worth trying, in order of implementation cost:

1. *Bin-extreme anchor for edge bins*: $a_0 = y_{\min}$, $a_{k-1} = y_{\max}$, $a_{\text{middle}} = c_i$. One-line change, crude, zero training cost.
2. *Empirical* $\widehat{\mathbb{E}}[y \mid p_i \ge 0.95]$: fit a classifier-only warmup, compute the mean of y among training samples with high p_i, and use that value as the anchor. More principled.
3. *Learnable soft anchor*: replace the hard pin with a parameter α_i initialised at c_i and add a soft penalty $\lambda\,(\alpha_i - c_i)^2$ — lets the head drift when data demands.
4. *Drop the anchor for edge bins*: keep it only for inner bins where the centroid is correct. Trades some identifiability for tail freedom.

What the anchor does and does not do. The anchor is a constraint on the **head function** h_i at $p_i = 1$, not a constraint on the model output μ_{total} for samples in bin i. At $p_i = 1$ exactly, $\mu_{\text{total}} = h_i(1)$; but for $p_i < 1$ (which is almost always the case — empirically $\max_x p_i(x)$ lands in 0.8–0.95, not 1.0), the prediction μ_{total} is a convex combination of all heads and differs freely from $h_i(1)$. For samples near the lower boundary of bin i we *want* $\mu_{\text{total}} < c_i$ via $p_{i-1} > 0$ and h_{i-1} leaking the mean down; the anchor does not prevent this. The anchor is a symmetry-breaking *limit* condition, not a pointwise constraint on the prediction.

3.4 How the three are orthogonal

- MDN changes the likelihood.
- CE_STOP_GRAD changes the supervision of the classifier.
- Anchoring changes the head's parametrization.

All combinations are meaningful, giving the 8-cell matrix below. ClassReg serves as a fifth ground-truth-bin-labels baseline.

4. Experimental design

4.1 The 8-cell matrix

cell	loss	optimization strategy	anchored heads
A	Gaussian LTV	`REGRESSION_ONLY`	no
B	Gaussian LTV	`REGRESSION_ONLY`	yes
C	Gaussian LTV	`CE_STOP_GRAD`	no
D	Gaussian LTV	`CE_STOP_GRAD`	yes

cell	loss	optimization strategy	anchored heads
E	MDN	`REGRESSION_ONLY`	no
F	MDN	`REGRESSION_ONLY`	yes
G	MDN	`CE_STOP_GRAD`	no
H	MDN	`CE_STOP_GRAD`	yes

Cell A is the pre-existing default. Cell D pins all three dials at once.

4.2 Datasets (all $n = 600$–800, $d = 1$)

1. **heteroscedastic**: $y = 2\sin(x) + 0.5x + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, (0.1 + 0.4|x|)^2)$, $x \sim \mathcal{U}(-5, 5)$. Smooth heteroscedastic noise; target range $\sim [-6, 6]$.
2. **bimodal**: $y = x + s \cdot 1.5 + \varepsilon$ with $s \sim \{-1, +1\}$, $\varepsilon \sim \mathcal{N}(0, 0.1^2)$, $x \sim \mathcal{U}(-3, 3)$. Two-mode posterior per input; tests the classification bottleneck.
3. **piecewise**: $y = 0.5x$ for $x < 0$, $y = 0.5x + \sin(4\pi x)$ for $x \geq 0$, plus Gaussian noise. Smooth-to-oscillatory transition; tests representational flexibility.
4. **exponential**: $y = e^x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.5^2)$, $x \sim \mathcal{U}(-3, 3)$. Heavy-tail, all-positive target in $[\sim 0, \sim 20]$.

4.3 Grid

- $k \in \{3, 5\}$
- 3 seeds per configuration (42, 43, 44)
- 80 epochs, LR 0.01, early stopping at 15 epochs, 20% val split
- 9 cells (ClassReg + 8 ProbReg cells) × 4 datasets × 2 k × 3 seeds = 216 trainings

4.4 Metrics (measured on test split)

- MSE(μ_{total}, y) — point-prediction error.
- Gaussian NLL — uses predicted point + Gaussian uncertainty from `predict_uncertainty()`.
- **anchor_error** (ProbReg only): $\max_i |h_i(p_i^\star) - c_i|$ where $p_i^\star$ is the test point at which class i is most confident. Measures how close each head is to its bin centroid at its own argmax — a direct probe of head–bin alignment.
- **max_p_mid** (at $k = 5$): $\max_x p_{\lfloor k/2 \rfloor}(x)$ — whether the middle class is ever the argmax. Low values mean the middle class is "swallowed" by its neighbours.

5. Raw results — full 8-cell × 4-dataset × 2-k table

Means over 3 seeds. Rows sorted by dataset, k, cell. **Best MSE per (dataset, k) group is highlighted by eye.** Bug-induced runs have been rerun; all numbers here are from clean training.

dataset	k	cell	MSE	MSE std	NLL (Gauss)	anchor_err	max_p_mid
bimodal	3	A	2.372	0.040	1.853	3.672	—
bimodal	3	B	2.397	0.079	1.858	0.030	—
bimodal	3	C	2.590	0.251	1.924	0.742	—
bimodal	3	D	2.589	0.212	1.921	0.110	—
bimodal	3	E	2.626	0.098	1.986	5.360	—
bimodal	3	**F**	**2.305**	0.039	1.851	1.801	—

dataset	k	cell	MSE	MSE std	NLL (Gauss)	anchor_err	max_p_mid
bimodal	3	G	2.728	0.250	2.003	0.529	—
bimodal	3	H	2.599	0.193	1.941	0.071	—
bimodal	3	ClassReg	3.350	0.083	2.030	—	—
bimodal	5	**A**	**2.329**	0.050	1.844	3.804	0.370
bimodal	5	B	2.379	0.055	1.856	0.223	0.289
bimodal	5	C	2.542	0.087	1.911	0.720	0.746
bimodal	5	D	2.677	0.048	1.926	0.316	0.755
bimodal	5	E	2.722	0.052	2.081	4.592	0.167
bimodal	5	F	2.382	0.148	1.866	1.377	0.210
bimodal	5	G	2.712	0.123	2.007	0.355	0.781
bimodal	5	H	2.788	0.142	2.046	0.348	0.744
bimodal	5	ClassReg	3.393	0.092	2.032	—	—
heteroscedastic	3	A	2.190	0.387	1.647	3.985	—
heteroscedastic	3	B	1.956	0.245	**1.484**	0.059	—
heteroscedastic	3	**C**	**1.775**	0.060	1.651	0.599	—
heteroscedastic	3	D	1.958	0.128	1.672	0.108	—
heteroscedastic	3	E	2.413	0.158	1.754	3.580	—
heteroscedastic	3	F	2.194	0.717	1.602	0.071	—
heteroscedastic	3	G	1.834	0.156	1.694	0.299	—
heteroscedastic	3	H	1.971	0.084	1.745	0.089	—
heteroscedastic	3	ClassReg	2.013	0.189	1.769	—	—
heteroscedastic	5	A	2.124	0.202	1.586	3.371	0.355
heteroscedastic	5	**B**	**1.803**	0.141	**1.486**	0.739	0.958
heteroscedastic	5	C	1.998	0.203	1.669	0.354	0.594
heteroscedastic	5	D	1.872	0.165	1.646	0.269	0.588
heteroscedastic	5	E	2.080	0.050	1.684	3.280	0.847
heteroscedastic	5	F	2.102	0.119	1.612	0.971	0.988
heteroscedastic	5	G	1.860	0.131	1.650	0.426	0.594
heteroscedastic	5	H	1.963	0.255	1.680	0.207	0.553
heteroscedastic	5	ClassReg	2.076	0.118	1.786	—	—
piecewise	3	A	0.334	0.089	0.621	3.363	—
piecewise	3	**B**	**0.321**	0.047	0.665	0.018	—
piecewise	3	C	0.329	0.014	0.789	0.427	—
piecewise	3	D	0.414	0.119	0.826	0.070	—
piecewise	3	E	0.378	0.154	0.710	3.346	—
piecewise	3	F	0.332	0.043	0.717	0.061	—
piecewise	3	G	0.428	0.139	0.875	0.238	—
piecewise	3	H	0.351	0.053	0.825	0.056	—
piecewise	3	ClassReg	0.372	0.051	0.926	—	—
piecewise	5	**D**	**0.284**	0.020	0.631	0.186	0.893
piecewise	5	A	0.293	0.050	0.532	2.382	0.050
piecewise	5	E	0.292	0.048	0.573	2.496	0.728
piecewise	5	G	0.292	0.023	0.710	0.347	0.923
piecewise	5	B	0.298	0.050	0.599	0.148	0.669
piecewise	5	F	0.298	0.031	0.624	0.207	0.944
piecewise	5	H	0.298	0.054	0.633	0.085	0.946
piecewise	5	C	0.311	0.030	0.624	0.352	0.927
piecewise	5	ClassReg	0.352	0.062	0.899	—	—
exponential	3	**E**	**0.507**	0.133	1.041	9.393	—
exponential	3	A	0.569	0.116	1.065	8.599	—
exponential	3	B	0.799	0.443	0.974	4.201	—
exponential	3	F	2.944	0.311	1.848	3.001	—
exponential	3	ClassReg	2.838	0.430	1.952	—	—

dataset	k	cell	MSE	MSE std	NLL (Gauss)	anchor_err	max_p_mid
exponential	3	D	5.489	0.612	1.561	0.177	—
exponential	3	G	5.691	0.544	1.682	0.225	—
exponential	3	C	5.769	0.599	1.604	0.448	—
exponential	3	H	5.782	0.633	1.710	0.187	—
exponential	5	**A**	**0.503**	0.189	1.028	17.789	0.083
exponential	5	E	0.995	0.543	1.220	16.775	0.214
exponential	5	B	1.836	0.366	1.133	1.278	0.610
exponential	5	D	1.860	0.430	1.234	0.151	0.740
exponential	5	C	1.876	0.698	1.358	1.117	0.726
exponential	5	H	2.127	0.499	1.444	0.327	0.742
exponential	5	G	2.263	0.655	1.406	0.249	0.746
exponential	5	F	2.295	0.316	1.414	1.104	0.968
exponential	5	ClassReg	2.571	0.216	1.901	—	—

Raw CSV: `automl_package/examples/probreg_identifiability_results/summary.csv`. Per-seed rows: `results.csv` in the same directory.

6. Visual interpretation — what the probability-curve pages show

The per-dataset PDFs (`results_*.pdf`) contain two families of plots: `h_i(p_i)` (head output as a function of its own probability) and `p_i(x)` (classifier probabilities vs input). The visual signatures give intuition that the numbers alone can hide.

6.1 `h_i(p_i)` plots — head-index anchoring

What to look for: at $p_i = 1$, does head i's output land on the dotted horizontal line at centroid c_i?

- **Unanchored Gaussian cells (A, C):** heads drift arbitrarily — bright crossings, heads swapping ordering between seeds. On bimodal k=3 for A, head 0 climbs from -1 to $+2$ while head 2 falls from -0.5 to -3. These are two valid assignments; the loss does not disambiguate.
- **Unanchored MDN cells (E):** the worst head-swap cases; on exponential E shows anchor_error = 9.4 ($k = 3$) and 17.8 ($k = 5$). MDN's per-component likelihood *should* identify which component goes where, but with only $n = 600$ samples and wide-range centroids the model does not converge to the canonical assignment from random init.
- **Anchored cells (B, D, F, H):** every head's output terminates exactly at its own centroid as $p_i \to 1$. Curves are flat near their anchor and only deviate for small p_i, where the residual f_i has scope. anchor_error drops to ≤ 0.2 in most cases.
- **ClassReg:** heads are flat horizontal lines at the centroid values (heads are the output of a lookup table, not a learned NN). Included as reference.

6.2 `p_i(x)` plots — classifier signature

- **ClassReg:** always the sharpest. Classes form crisp, almost step-like partitions of x-space — the classifier has been trained on hard bin labels and commits to a decision boundary.
- **Cells G, H (MDN + CE_STOP_GRAD):** look nearly identical to ClassReg. This is the second visual confirmation of the identifiability claim: once the classifier is supervised by bin-CE, the probability curves become classifier-like regardless of the regression loss family.
- **Cells A, B (Gaussian + REGRESSION_ONLY):** softer, heavily overlapping curves. The regression loss tolerates diffuse p as long as μ_{total} is right — so the classifier never commits hard.

- **Cell E, F (MDN + REGRESSION_ONLY):** noisier, sometimes with one class collapsed (low $\max p$). MDN can trade off component weights freely, producing unstable curves on wider-range data.
- **Cells C, D on exponential (and only there):** classifier curves look classifier-like (as expected from CE_STOP_GRAD), but the regression heads behind them diverge from true bin means — see §8 for the mechanism.

6.3 What `max_p_mid` tells us at $k = 5$

Recall the middle class is $\lfloor k/2 \rfloor = 2$. `max_p_mid` is the maximum probability ever assigned to class 2 across test x. Low values mean the middle class is essentially unused.

- max_p_mid < 0.3: middle class is swallowed by its neighbours (seen in A and E, which lack any mechanism to keep it alive).
- max_p_mid ≥ 0.6: middle class actively used for some inputs. Anchored and CE_STOP_GRAD cells land here.

This is consistent with anchored heads and bin-CE supervision both providing structural pressure for every class to be meaningful.

7. Analysis and conclusions

7.1 Anchored heads — recommend ON by default

Anchoring is the cleanest win. Across all 4 datasets $\times$ 2 k values:

- anchor_error drops from ~3–9 (unanchored) to 0.02–0.4 (anchored).
- MSE cost is $\leq 5\%$ on 3/4 datasets.
- NLL cost is negligible.
- Visually: the head-swap degeneracy disappears entirely.
- Structural: $h_i(1) = c_i$ is a hard guarantee, not statistical.

Recommendation: flip `use_anchored_heads=True` as the default parametrization.

7.2 ClassReg vs ProbReg

ClassReg is consistently beaten by at least one ProbReg cell on all four datasets:

dataset	best ProbReg MSE	ClassReg MSE	gap
bimodal k=3	2.305 (F)	3.350	−31%
heteroscedastic k=3	1.775 (C)	2.013	−12%
piecewise k=3	0.321 (B)	0.372	−14%
exponential k=3	0.507 (E)	2.838	−82%

The gap is largest on bimodal and exponential, where a smooth posterior is critical. ClassReg's step-function classifier cannot interpolate between bins; ProbReg's soft probabilities can. On piecewise, where the target is nearly deterministic, the gap is small.

7.3 CE_STOP_GRAD — situational dial, not default

- Wins heteroscedastic MSE (Cell C = 1.775, beating the unanchored Gaussian A = 2.190 by 19%).
- Loses bimodal MSE (Cell C = 2.590 vs Cell A = 2.372, +9%).
- **Catastrophically fails on exponential** (MSE $\sim$ 5.5-5.8 across C, D, G, H vs ≤ 0.8 for A, B, E). See §8 for why.
- At $k = 5$, CE_STOP_GRAD cells scale worse — Cell C heteroscedastic goes 1.775 $\rightarrow$ 1.998 while Cell B goes 1.956 $\rightarrow$ 1.803.

CE_STOP_GRAD is best kept as an opt-in dial for bounded / symmetric targets where classifier semantics are wanted.

7.4 MDN NLL — situational, not uniform

- Wins exponential k=3 (Cell E = 0.507, best overall on that row).
- Wins bimodal k=3 (Cell F = 2.305, beats A by $\sim$ 3%).
- Loses on heteroscedastic and piecewise (Gaussian LTV wins both).
- MDN + anchored is the most robust MDN choice.

Keep Gaussian LTV as the default loss; expose MDN as a `prob_reg_loss_type` dial for bimodal / heavy-tail data.

7.5 Recommendation matrix

setting	default	expose as dial	rationale
anchored heads	**on**		free identifiability, no MSE cost
Gaussian LTV vs MDN	Gaussian	yes	MDN is situationally better, not uniformly
`REGRESSION_ONLY` vs `CE_STOP_GRAD`	`REGRESSION_ONLY`	yes	CE_STOP_GRAD breaks on heavy-tail targets
ClassReg vs ProbReg	**ProbReg**		ClassReg only competitive on narrow-range targets

7.6 The best "safe default" overall

Cell B — Gaussian LTV + `REGRESSION_ONLY` + anchored heads. It either wins or ties-best on NLL across all four datasets and is the highest among cells that do not fail catastrophically on any. If classifier semantics are specifically wanted (e.g. calibration / interpretability applications where we need $p_i(x)$ to mean "probability of bin i"), **Cell D** is the right choice at the cost of mild MSE degradation outside exponential.

8. The symlog hypothesis — a negative result

8.1 The diagnosis we proposed

On exponential ($y \in [\sim 0.05, \sim 20]$), percentile bins put $k = 3$ centroids roughly at $c = (0.2, 1.5, 12)$. Their spread is enormous. Evaluating the LTV between-class variance at uniform $p = (1/3, 1/3, 1/3)$ and with head outputs initialised near their centroids:

$$\underbrace{\tfrac{1}{3}(0.04 + 2.25 + 144)}_{\approx 48.8} - \underbrace{(\tfrac{1}{3}(0.2 + 1.5 + 12))^2}_{\approx 20.9} \approx 27.9.$$

So $\sigma^2_{\text{total}} \gtrsim 28$ at initialisation. The Gaussian NLL gradient with respect to any head's mean at a sample y is

$$\frac{\partial \mathcal{L}_{\text{LTV}}}{\partial \mu_j} \propto -\frac{p_j\,(y - \mu_{\text{total}})}{\sigma^2_{\text{total}}}.$$

With $\sigma^2_{\text{total}} \approx 28$, a residual of magnitude ~ 3 produces gradient contributions $\sim 0.1 \cdot p_j$ — an order of magnitude smaller than on bimodal ($\sigma^2_{\text{total}} \approx 2$). So heads receive weak signal.

Under `REGRESSION_ONLY`, the regression loss can still adapt the classifier to collapse p toward a single class, which reduces σ^2_{total} quickly and unblocks the head gradient. Under `CE_STOP_GRAD`, the classifier is pinned by bin-CE and cannot compress p; the heads stay stuck on this "flat loss plateau" through training. This would explain the 10× MSE catastrophe on exponential for C/D/G/H.

Predicted fix: `target_transform="symlog"` compresses y to $\text{symlog}(y) = \text{sign}(y)\log(1 + |y|) \in [\sim 0.05, \sim 3.04]$. In that space, centroids are roughly $(0.18, 0.92, 2.56)$ and the between- class variance drops to ~1.1 — comparable to bimodal. Predicted outcome: heads recover proper gradient and C/D converge.

8.2 The experiment

Script: `automl_package/examples/exponential_symlog_probe.py`. 96 trainings = 8 cells × 2 k × 2 transforms × 3 seeds on exponential. Results at `automl_package/examples/exponential_symlog_probe_results/`

8.3 Results

MSE, mean over 3 seeds:

cell	k	none	symlog	verdict
A	3	0.69	0.79	baseline, unaffected
B	3	1.91	2.05	slight regression
C	3	**4.43**	**4.49**	**no improvement**
D	3	**4.08**	**4.12**	**no improvement**
E	3	0.50	1.67	symlog *hurts* unanchored MDN
F	3	1.85	**0.40**	**5× improvement** (MDN + anchored)
G	3	4.17	4.21	~same
H	3	4.70	4.07	marginal
A	5	0.40	0.47	baseline
C	5	2.78	2.67	marginal
D	5	1.90	2.06	slight regression

cell	k	none	symlog	verdict
F	5	1.92	1.23	moderate improvement

8.4 What it rules out and what it leaves

The target-range-compression story is **not** the dominant cause. C/D stay broken on exponential under symlog by margins indistinguishable from the original training. The inter-head variance term is real math, but either (a) it is not the binding constraint in practice, or (b) the classifier under CE_STOP_GRAD is itself not learning well enough on exponential *regardless* of target scale.

Surprise finding: symlog rescues Cell F (MDN + anchored) by 5× on exponential k=3. This is not what we predicted to improve. Our best explanation: MDN per-component likelihoods are sensitive to target scale (the $\log \sigma_i$ term grows with range), and symlog removes that pressure. Anchored heads absorb the residual head-swap risk MDN would otherwise bring back. This combo is interesting for heavy-tail real applications (photo-z, cluster mass).

8.5 Remaining suspects for the C/D failure

1. **Percentile bins on heavy-tail** y: bin $k-1$ on exponential spans $[3, 20]$ — a single mean per bin is a terrible fit. Under REGRESSION_ONLY, regression loss can compress p to shift mass toward specific samples within the bin and still fit them tolerably (because μ_{total} is a mixture). Under CE_STOP_GRAD, each head must commit to a single bin-mean value and is punished everywhere else in the bin. This would explain why larger k helps (C $k = 5 = 2.78$ vs $k = 3 = 4.43$ — halves with finer bins).
2. **No classifier/head co-adaptation** under CE_STOP_GRAD means the training regime is a two-player non-cooperative game rather than a joint-optimization — heads get worse gradient statistics.
3. **Head initialization**: anchored heads start at centroids (near correct), unanchored heads at 0 (far from the 12 centroid). Yet anchored cells D/H still fail — ruling out "bad init" as the primary cause and re-pointing at bin/target geometry.
4. **Anchor target mis-specification** (see §3.3): pinning $h_i(1) = c_i$ is wrong for edge bins, because $\mathbb{E}[y \mid p_i(x) = 1]$ does not equal c_i when the bin is asymmetric about its mean. On exponential's upper bin, the correct anchor is well above c_{k-1}, so the current anchor forces a systematic under-prediction on the upper tail. This is the most likely root cause of the anchored cells (B, D, F, H) still failing on exponential and is a concrete actionable fix — worth trying *before* the hybrid opt-strategy (§9.1).

9. Open questions and next experiments

In priority order.

9.1 Hybrid opt-strategy (most promising)

Train jointly (`REGRESSION_ONLY`) for the first ~30 epochs, then switch to `CE_STOP_GRAD` for the remainder. Gives classifier + heads time to co-adapt before locking the classifier. Would validate or refute suspect (2) in §8.5.

Implementation sketch: add `ProbabilisticRegressionOptimizationStrategy.HYBRID` with a `ce_stop_grad_after_epoch: int = 30` knob in `probabilistic_regression.py`. At inference-time

the heads see detached probs; the classifier stays frozen on bin-CE post-switch.

9.2 Adaptive bin boundaries on heavy-tail y

Replace percentile bins (which widen at the heavy tail) with uniform-on- symlog-y bins on wide-range targets. Would test suspect (1).

9.3 Per-head log-variance warm start

Initialise each head's $\log \sigma_i^2$ bias from $\log \mathrm{Var}[y \mid y \in \mathrm{bin}_i]$. Makes the NLL well-scaled from step 1 even when the classifier is pinned. Cheapest intervention; should test before (9.1).

9.4 Anchored + MDN + symlog on real domains

The Cell F + symlog combo was the most dramatic improvement we've seen. Test it on photo-z (targets span orders of magnitude in galaxy redshift) and cluster mass (log-normal mass function). If it generalises, it's a candidate Paper A configuration specifically for heavy-tail astrophysics.

9.5 Re-test Cell D with classifier warm-start

Train the classifier alone on bin-CE for 10 epochs (no regression loss active), then flip regression loss on with `CE_STOP_GRAD`. Should reveal whether the classifier is the bottleneck or the heads are.

9.6 Larger k sweep on exponential

Already partial evidence that $k = 5$ halves C's MSE from $k = 3$. Run $k \in \{3, 5, 10, 20\}$ on exponential to confirm the trend and find the turn-over where intra-bin variance is negligible.

10. Bug log (during this investigation)

Four bugs were found and fixed (three new, one documented-but-unfixed) while running this sweep. Durable record here so future runs don't repeat them.

10.1 SINGLE_HEAD_FINAL_OUTPUT + dynamic-k shape mismatch (`280ad1f`)

`_compute_predictions_for_k` had a special branch for `SINGLE_HEAD_FINAL_OUTPUT` that applied a weighted sum over per-class outputs, but that strategy already returns `(batch, output_size)` — the weighted sum caused a shape mismatch. Crashed all dynamic-k ProbReg runs with this strategy.

Affected: 12/48 configs of `probreg_ablation` (first overnight run). Reran after fix; clean now.

10.2 `calculate_class_value_ranges` numpy.float32 rejection (`19bdacc`)

PyTorch 2.6+ refuses to assign `numpy.float32` scalars to tensor elements via `tensor[i, j] = scalar`. Caller passes `np.min(y)` which returns a `numpy.float32` when y is `float32`.

Affected: all 50 `ProbReg+HPO` trials on UCI-Yacht in the original HPO sweep (produced NaN across the board because boundary regularisation was in the Optuna search space). Rerun after fix.

10.3 BatchNorm with batch size 1 (`fe63c90`)

UCI-Yacht has 308 samples; after the 0.3 test split and internal 0.1 test / 0.2 val splits, the ProbReg HPO final training saw $n_{\text{train\_val}} = 193$ samples. With `batch_size=32`, $193 \bmod 32 = 1$ — the last batch contains a single sample, and `BatchNorm` in training mode refuses batches of size 1.

Fix: `drop_last=True` when the last batch would have exactly 1 sample. Triggered only under HPO on UCI-Yacht; other sweeps use datasets whose sizes do not produce a 1-sample remainder.

10.4 `final_results_report.py` column name (`fe63c90`)

The aggregator expected a column `nll_mean` but the identifiability sweep CSV produces `nll_own_mean`, `nll_gaussian_mean`, `nll_mdn_mean`. Fixed to tolerate either.

10.5 B1 — monotonic head init with `mean=-3.0` (pre-existing, not fixed)

`_initialize_monotonic_head` uses `nn.init.normal_(weight, mean=-3.0)`, which breaks on all-positive targets. **Does not affect this investigation** — the identifiability sweep sets `use_monotonic_constraints=False`. Affects `head_degeneracy_diagnostic` and `overfitting_showcase` on exponential; their exponential results should be disregarded until B1 is fixed.

11. File, artefact, and commit pointers

Scripts

purpose	path
main sweep	`automl_package/examples/probreg_identifiability_swee`
symlog probe	`automl_package/examples/exponential_symlog_probe.py`
aggregator PDF	`automl_package/examples/final_results_report.py`
implementation plan	`docs/probreg_identifiability_implementation_plan.md`

Result CSVs

purpose	path
per-seed results	`automl_package/examples/probreg_identifiability_resu`
cross-seed summary	`automl_package/examples/probreg_identifiability_resu`
symlog probe	`automl_package/examples/exponential_symlog_probe_resu`

PDFs (all under `automl_package/examples/probreg_identifiability_results/`)

- `results_bimodal.pdf` — metrics table + head curves + probability curves
- `results_heteroscedastic.pdf`
- `results_piecewise.pdf`
- `results_exponential.pdf`

Merged master PDF: `automl_package/examples/final_results_report/final_report.pdf` (concatenates the sweep-level summary with every per-sweep PDF; 38 pages via `pdfunite`).

Key commits

- `280ad1f` — fix: SINGLE_HEAD_FINAL_OUTPUT dynamic-k shape mismatch
- `19bdacc` — fix: numpy.float32 in calculate_class_value_ranges
- `fe63c90` — fix: BatchNorm batch_size=1 + report column name
- `4d7d9fe` — final_results_report: merge per-sweep PDFs via pdfunite
- `7b92ead` — restore GRADIENT_STOP semantics, RegressionHead ABC

12. What we'd pick up first in a future session

1. Implement the **hybrid opt-strategy** (§9.1). This is the most likely source of insight about whether CE_STOP_GRAD on exponential is a co-adaptation failure or something else.
2. Set `use_anchored_heads=True` as the **ProbReg default** and remove the `constrain_middle_class` branch that it subsumes.
3. Run the **anchored + MDN + symlog** combination on photo-z / cluster-mass real data (§9.4) — our one dramatic positive surprise and the combination most likely to matter for Paper A.
4. Add `ce_stop_grad_after_epoch` experiments into the main ablation sweep infrastructure so these numbers feed Paper A naturally.